

Unit 2 Study Guide

Semantic HTML and Forms

9 topics · condensed key concepts · exam-style practice questions with answers

How to use this guide

Part 1 is a condensed, topic-by-topic reference — read it as a quick refresher, not a first-time introduction (for that, use the full tutorials on the site). Part 2 is a set of exam-style practice questions covering the whole unit, in several formats you're likely to see on a real exam: short answer, differentiate-between, write-the-code, fill-in-the-blank, and true/false. Answers are collected at the end so you can self-check without seeing them while you attempt the questions.

by Dr. Puneet Thapar — Associate Professor, School of Computing and Artificial Intelligence, Lovely Professional University

Part 1 — Quick Reference

17. Semantic HTML Elements

- Semantic elements describe meaning, not just appearance — `<div>/<span>` carry no meaning; `<header>/<nav>/<article>` do.
- `<header>`, `<nav>`, `<main>`, `<footer>` mark major page regions, exposed to screen readers as landmarks.
- `<article>` is self-contained/redistributable content; `<section>` groups thematically related content under its own heading; `<aside>` is tangentially related content.
- `<time datetime="...">` marks a machine-readable date; `<address>` marks contact info; `<details>/<summary>` build a native disclosure widget.

18. Page Structure and Layout

- A full page composes header + nav + main (with section/article/aside) + footer into one coherent structure.
- Semantic elements map automatically to ARIA landmark roles (header→banner, nav→navigation, main→main, aside→complementary, footer→contentinfo).
- A "Skip to main content" link, visible on focus, lets keyboard/screen-reader users bypass repeated navigation.
- HTML structure describes meaning; CSS (Flexbox/Grid) decides visual arrangement independently.

19. Accessibility Fundamentals

- Accessibility follows the POUR principles: Perceivable, Operable, Understandable, Robust.
- Every image needs alt — descriptive for meaningful images, empty `alt=""` for purely decorative ones.
- Every interactive control must be reachable/usable by keyboard alone; avoid positive `tabindex` values; never remove focus outline without replacing it.
- Don't use ARIA if a native HTML element already provides the needed semantics ("no ARIA is better than bad ARIA").
- Never rely on color alone to convey meaning; ensure sufficient contrast.

20. Forms and Form Controls

- `<form action="..." method="get/post">` wraps controls submitted together; `enctype="multipart/form-data"` is required for file uploads.
- Core controls: `<input>`, `<textarea>`, `<select>/<option>`, `<button>`.
- Every input needs a `<label>` (via matching `for/id`, or by wrapping) — required for accessibility and larger click targets.
- `<fieldset>+<legend>` groups related controls (especially radio/checkbox groups).
- Inside a `<form>`, a plain `<button>` defaults to `type="submit"` unless set otherwise.

21. Input Types and Attributes

- Text-based types: email, password, search, tel, url — each brings the right mobile keyboard and basic validation.
- Numeric/date types: number, range, date, time, month, week, `datetime-local`.
- Selection/file types: checkbox, radio, color, file.
- Shaping attributes: placeholder, required, disabled, readonly, min/max/step, minlength/maxlength, pattern, autocomplete, autofocus.

22. Form Validation Basics

- Built-in constraint validation (required, pattern, min/max, minlength/maxlength, type checks) is enforced by the browser with zero JavaScript.
- `:valid` / `:invalid` CSS pseudo-classes hook into a field's live validation state.
- `setCustomValidity()` (small JS) replaces the browser's default error message with custom wording.

- novalidate on <form> disables native validation entirely, for pages needing fully custom JS-driven validation UI.

23. Tables and Data Representation

- <table> / <tr> / <td> / <th> are the basic row-and-cell structure.
- <thead>/<tbody>/<tfoot> group rows for structure, styling, and repeated printing of long tables.
- <caption> titles the whole table (first child after <table>); colgroup/col style whole columns at once.
- colspan/rowspan merge cells across columns/rows; scope="col"/"row" on <th> makes tables accessible.

24. Accessibility Standards

- WCAG (Web Content Accessibility Guidelines) organizes testable success criteria around the four POUR principles.
- Conformance levels: A (minimum), AA (most common legal/organizational baseline), AAA (highest).
- ADA (US), Section 508 (US federal), and EN 301 549 (EU) are policies that reference WCAG — general context, not legal advice.
- Automated tools (axe, WAVE, Lighthouse) catch only ~30–40% of real issues; manual keyboard/screen-reader testing matters too.

25. Responsive Content Structuring

- The viewport meta tag (width=device-width, initial-scale=1.0) is the foundation of responsive design — without it, mobile browsers zoom out a desktop layout.
- Percentages, minmax(), and CSS Grid's auto-fit/auto-fill let a layout reflow its own column count.
- Mobile-first: base styles target small screens, then min-width media queries layer on complexity for larger ones.
- Flexbox's order property changes visual order without changing HTML source order (which screen readers/tab order still follow).

Part 2 — Exam Practice Questions

Section A — Short Answer

- Q1. What is the difference in meaning between `<section>` and `<div>`?
- Q2. Explain the POUR principles of web accessibility.
- Q3. Why is a "Skip to main content" link useful, and who benefits most from it?
- Q4. What HTML attribute must a `<form>` use when it contains a file upload, and why?
- Q5. Why must every form `<input>` have an associated `<label>`?
- Q6. What does the `required` attribute do, and does it need any JavaScript to work?
- Q7. What is the purpose of the `<caption>` element in a table?
- Q8. What is the difference between WCAG conformance levels A, AA, and AAA?
- Q9. Why do automated accessibility testing tools alone not guarantee a fully accessible page?
- Q10. What problem does the viewport meta tag solve on mobile devices?

Section B — Differentiate Between

- Q11. [Differentiate] `<article>` vs. `<section>` vs. `<aside>`
- Q12. [Differentiate] `colspan` vs. `rowspan`
- Q13. [Differentiate] `:valid/invalid` vs. `novalidate`
- Q14. [Differentiate] `tabindex="0"` vs. `tabindex="-1"` vs. a positive `tabindex` value
- Q15. [Differentiate] Mobile-first vs. desktop-first media query strategy

Section C — Write the Code

- Q16. [Code] Write a `<nav>` element with an `aria-label` distinguishing it from other navigation regions.
- Q17. [Code] Write a labeled email `<input>` that is required, using an explicit `for/id` pairing.
- Q18. [Code] Write a `<fieldset>/<legend>` grouping two radio buttons for a "shipping speed" choice.
- Q19. [Code] Write a table with `<thead>`, `<tbody>`, and one `<th scope="col">` header cell.
- Q20. [Code] Write the viewport meta tag exactly as it should appear in `<head>`.

Section D — True or False

- Q21. [T/F] A page should have more than one visible `<main>` landmark.
- Q22. [T/F] `aria-hidden="true"` should be used on decorative icons that add no information.
- Q23. [T/F] Inside a form, a `<button>` with no `type` attribute defaults to `type="button"`.
- Q24. [T/F] `scope="col"` on a `<th>` helps screen readers associate that header with its column.
- Q25. [T/F] Level AA is generally considered the common baseline for WCAG conformance.

Answer Key

- A1.** `<section>` groups thematically related content and should normally have its own heading; `<div>` is a purely generic container with no semantic meaning at all.
- A2.** Perceivable (content must be presentable in ways users can perceive), Operable (interface components must be operable, e.g. by keyboard), Understandable (content and operation must be understandable), Robust (content must work reliably across current and future assistive technologies).
- A3.** It lets a keyboard or screen-reader user jump straight past a long, repeated navigation menu to the page's unique content, instead of tabbing through the whole menu on every single page.
- A4.** `enctype="multipart/form-data"` — required so the browser can correctly send binary file content as part of the form submission.
- A5.** Labels are announced by screen readers (without one, a field has no accessible name) and clicking a label focuses/checks its control — important for touchscreens and larger click targets.
- A6.** `required` marks a field as mandatory and blocks form submission until it's filled in — enforced entirely by the browser, no JavaScript needed.
- A7.** `<caption>` gives the whole table a title/summary, which screen readers announce immediately after entering the table, giving context before the data itself.
- A8.** A (minimum, most basic accessibility), AA (mid-range, the most common legal/organizational target), AAA (highest, often impractical to achieve for every piece of content).
- A9.** Automated tools are estimated to catch only around 30–40% of real accessibility issues — things like an illogical focus order or a confusing accessible name require manual keyboard/screen-reader testing to catch.
- A10.** Without it, mobile browsers assume a wide desktop layout and zoom the whole page out to fit, making text and controls tiny and hard to use.
- B1.** `<article>` is self-contained, independently distributable content (a blog post); `<section>` groups thematically related content under its own heading; `<aside>` holds tangentially related content, like a sidebar.
- B2.** `colspan` merges a cell across multiple columns (horizontally); `rowspan` merges a cell down multiple rows (vertically).
- B3.** `:valid/invalid` are CSS pseudo-classes that style a field based on its live constraint-validation state; `novalidate` is an HTML attribute on `<form>` that disables the browser's automatic validation entirely.
- B4.** `tabindex="0"` makes an element focusable in normal document order; `tabindex="-1"` makes it focusable only via JavaScript (skipped by Tab); a positive value forces a custom tab order, which is discouraged since it usually confuses users.
- B5.** Mobile-first writes base styles for small screens first, then adds min-width media queries to layer on complexity for larger screens; desktop-first does the reverse, starting wide and using max-width queries to simplify downward.
- C1.** `<nav aria-label="Primary"><a href="/">Home</a></nav>`
- C2.** `<label for="email">Email</label><input id="email" type="email" required>`
- C3.** `<fieldset><legend>Shipping speed</legend><label><input type="radio" name="speed" checked> Standard</label><label><input type="radio" name="speed"> Express</label></fieldset>`
- C4.** `<table><thead><tr><th scope="col">Name</th></tr></thead><tbody><tr><td>Value</td></tr></tbody></table>`
- C5.** `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
- D1.** False — there should normally be exactly one visible `<main>` per page.
- D2.** True.
- D3.** False — a `<button>` with no type inside a form defaults to `type="submit"`.

D4. True.

D5. True.